

Concetti Generali

Il progetto software comprende sia il processo di progettazione sia il risultato finale della progettazione.

- La progettazione è il processo attraverso cui vengono definite:

- l'architettura del sistema,
- le componenti,
- le interfacce,
- e le altre caratteristiche del software.

- Il progetto dell'applicazione è invece il prodotto finale di questa attività, cioè il modello progettuale che guiderà implementazione e test.

Ruolo del Progetto

Permette allo sviluppatore di analizzare diverse soluzioni prima di costruire realmente il software, valutare vari modelli progettuali e scegliere quello che rappresenta il miglior compromesso tra:

- conformità ai requisiti,
- qualità del sistema,
- costi,
- facilità di manutenzione,
- prestazioni.

Una volta scelto il modello più adatto, esso viene utilizzato come guida per le successive attività di codifica e testing.

Pratiche di Progettazione

Rappresentano le linee guida che permettono di applicare concretamente i principi dell'ingegneria del software.

Una delle pratiche più importanti è:

- Decomposizione e Modularità = suddivisione di un sistema complesso in moduli più piccoli e il più indipendenti possibili
Ogni modulo contiene specifiche funzionalità o responsabilità come:
 - Coesione ed accoppiamento: i moduli devono essere ben coesi tra loro per un alto legamento di componenti interne ed un basso accoppiamento cioè essere il più indipendenti possibile.
- Generalizzazione = consiste nel risolvere un problema in modo generale da poter essere riutilizzato anche in situazioni simili

la parametrizzazione permette di:

- generalizzare le procedure,
- generalizzare i dati,
- generalizzare il controllo e le interazioni.

Questo approccio aumenta enormemente la riusabilità del software.

- Incapsulamento ed information hiding = consistono nel raggruppare dettagli interni e decisioni implementative dentro specifiche componenti del sistema
 - Qualunque esigenza di cambiamento deve essere localizzata in una componente
 - I cambiamenti iniettati in una componente devono ripercuotersi in poche altre
 - Questa è una pratica che sintetizza i principi di Separazione degli Interessi e di Astrazione
- Anticipazione dei cambiamenti: predisposizione ad accogliere cambiamenti, generati da una qualsiasi causa, tempestivamente con poco impegno e basso rischio
 - Sufficienza, completezza e ricerca di primitive = ogni modulo deve contenere tutte e sole le caratteristiche primitive necessarie alla propria astrazione evitando moduli incompleti e inutili.

MODULARITA'

Componente software che fornisce specifici servizi o funzionalità. Un intero sistema software può essere visto come un insieme di moduli che collaborano tra loro.

Moduli e Relazioni

I moduli collaborano tra loro con relazioni ben definite permettendo ai moduli di utilizzare servizi forniti da altri garantendo stabilità e comprensibilità.

La Relazione USES

A USES B

- Perché A operi correttamente è necessario che B operi correttamente
- A accede ai servizi esportati da B attraverso la sua interfaccia
- A dipende da B per fornire i suoi servizi

A è un client di B e B è un server di A

La relazione USES è statica, cioè indipendente dall'esecuzione concreta del software.

Proprietà desiderabili

La relazione USES dovrebbe essere gerarchica, cioè organizzata come un grafo orientato aciclico. Questo rende il software:

- più semplice da comprendere,
- più semplice da validare,
- più facile da mantenere,
- più riusabile.

Se invece esistono dipendenze cicliche, si rischia di avere sistemi in cui “nulla funziona finché tutto non funziona”.

Fan-in e Fan-Out

- Fan-in → numero di moduli che usano un modulo;
- Fan-out → numero di moduli usati da un modulo.

Una buona struttura software dovrebbe avere:

- fan-in alto → il modulo viene riutilizzato molto;

- fan-out basso → il modulo dipende da pochi altri moduli.

Le interfacce nella relazione USES

Bisogna sapere cosa esporta un modulo usato attraverso la sua interfaccia.

Il modulo client conosce solo ciò che il server esporta attraverso la propria interfaccia, mentre i dettagli implementativi rimangono nascosti.
L'interfaccia

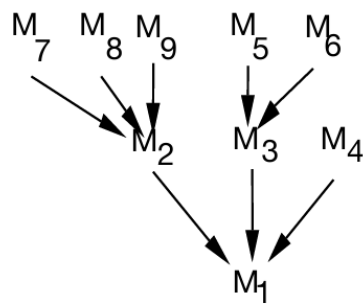
- agisce come un contratto tra un modulo ed i suoi client
- essa è anche l'astrazione delle capacità implementate nel server

Questa è una pratica del principio di separazione degli interessi.

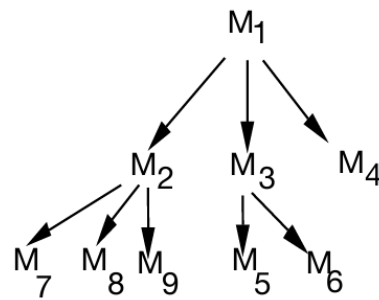
Le relazioni: Is-Component-Of e Comprises

Queste relazioni descrivono la gerarchia strutturale del sistema.
“A Is-Component-Of B” significa che A è una componente di B.
“B Comprises A” è la relazione reciproca.

Queste relazioni permettono di rappresentare il sistema come una gerarchia di componenti a diversi livelli di astrazione.



(Is-Component-Of)



(Comprises)

Entrambe descrivono la stessa **gerarchia**

INFORMATION HIDING

Rendere possibile il cambiamento del comportamento di una componente senza conoscere il comportamento delle altre.

- le decisioni progettuali devono essere nascoste dentro moduli specifici;
- i dettagli instabili devono essere isolati;
- dettagli indipendenti devono essere separati;
- le interfacce devono rimanere stabili anche se cambia l'implementazione interna.

Valutazione della Stabilità

Dipende dalla capacità del progettista di prevedere:

- il dominio applicativo,
- le piattaforme hardware e software,
- le future evoluzioni del sistema.

Un progettista esperto riesce quindi a progettare componenti più stabili e meno soggette a modifiche frequenti.

- Individuare i seguenti tipi di componenti:
 - HW/SW hiding : modificate solo quando è cambiata la macchina virtuale

Nasconde dettagli legati:

- all'accesso ai dati,
- ai dispositivi di input/output,
- ai sistemi di comunicazione.

In questo modo eventuali cambiamenti hardware o software non influenzano direttamente il resto dell'applicazione.

- Behavior hiding : devono essere cambiate quando si modificano le specifiche del sistema

nasconde:

- algoritmi,
- logiche applicative,
- modalità di interazione,
- processi procedurali.

Questo permette di modificare il comportamento interno senza alterare il resto del sistema.

- Decision hiding : modificate quando cambiano le decisioni sulle tecniche utilizzate e sulle caratteristiche SW richieste.

nasconde:

- strutture dati,
- algoritmi,
- parametri di configurazione,
- politiche implementative.

Lo scopo è isolare le decisioni progettuali che potrebbero cambiare nel tempo.

PROCESSO DI PROGETTAZIONE SOFTWARE

Il processo è iterativo e ogni fase contribuisce alla costruzione del progetto definitivo.

Manufatti e Strategie di Produzione

I principali manufatti della progettazione sono:

- Architettura → organizzazione del sistema;
- Progetto dettagliato → comportamento delle componenti e la loro collaborazione.

strategie progettuali:

- *Generali*: dividi-e-conquista; raffinamento per passi; top-down vs bottom-up; astrazione; information hiding; euristiche, pattern; incrementi successivi;
- *Funzionali*: basato sulla individuazione delle funzioni principali.
- *Orientata agli Oggetti*: basato sui dati ed i metodi che ne sono proprietari
- *Centrata sulle strutture di dati*: basato sulle strutture di dati che esso deve gestire piuttosto che sulle funzioni
- *Basata sulle componenti*: la strategia prevede di individuare le componenti necessarie in un software per aumentare il riuso. Componenti con un certo livello di qualità non devono dipendere da componenti o servizi con minore livello di qualità.

Strutture Software utilizzate nella Progettazione

- Stili Architetturali: Un insieme di vincoli per esprimere un'architettura che definisce una famiglia di progetti
- Pattern di Progetto: Soluzioni comuni per un problema comune
- Framework: Una soluzione generale che deve essere specializzata e può essere estesa aggiungendo qualche specifico plug-in
- Linee di prodotto: Soluzione che prevedono parti invarianti comuni a tutti i prodotti della linea e parti varianti che specificano ogni singolo prodotto della linea

ARCHITETTURA

L'architettura deve essere progettata non solo per soddisfare i requisiti attuali, ma anche per adattarsi economicamente ai requisiti futuri.

L'architettura deve inoltre affrontare una serie di problemi tipici dei sistemi software:

- Concorrenza, cioè la presenza di più attività eseguite contemporaneamente.
- Controllo degli eventi, cioè i sistemi software spesso devono reagire a eventi esterni o interni.
- Distribuzione e deployment, cioè come il software viene distribuito sull'hardware disponibile.
- Rilevazione e trattamento degli errori e delle eccezioni
- Interattività del sistema, cioè il modo in cui il software comunica con utenti o altri sistemi.
- Dati condivisi e della persistenza cioè alcuni dati devono essere accessibili da più moduli senza compromettere la manutenibilità.
- Persistenza cioè i dati devono vivere

PROGETTO DI DETTAGLIO

Rappresenta il passaggio dall'architettura astratta all'implementazione concreta del sistema.

Sottolineando che una stessa architettura può essere realizzata in molti modi diversi, quindi il progetto di dettaglio serve a scegliere la soluzione specifica che verrà realmente implementata.

Per costruire il progetto dettagliato bisogna:

- identificare le componenti,
- definire il comportamento di ogni componente,
- stabilire come le componenti collaborano,

- definire le regole di composizione,
- definire la semantica del sistema.

Le componenti possono essere:

- elementari,
- composte,
- sottosistemi completi.

Tipi di componenti:

Le **Black Box** sono componenti di cui non si conosce l'implementazione interna. Si conoscono solo:

- interfaccia,
- input,
- output,
- servizi offerti.

Le **White Box** sono componenti completamente trasparenti: si conosce tutta l'implementazione interna. In questo caso l'interfaccia rappresenta solo una parte dei metodi disponibili. Questo approccio offre massimo controllo ma aumenta anche la dipendenza dall'implementazione.

Le **Grey Box** o **Glass Box** rappresentano una situazione intermedia. Il codice è disponibile e consultabile, ma non necessariamente modificabile.

INTERFACCE UOMO MACCHINA ed IMPLEMENTAZIONE

Il modo in cui l'utente interagisce con il sistema software. L'interfaccia rappresenta il punto di contatto tra essere umano e applicazione.

Principi per l'interfaccia utente

- Apprendibilità: l'interfaccia deve essere facile da imparare. L'utente può tempestivamente mettersi a lavorare
- Familiarità: l'interfaccia deve utilizzare termini, simboli e concetti familiari all'utilizzatore.
- Consistenza: significa che azioni simili devono richiedere operazioni simili. Se una certa operazione viene eseguita in un modo in una schermata, dovrebbe funzionare nello stesso modo anche nelle altre parti del sistema.
- Recuperabilità: l'interfaccia deve permettere all'utente di recuperare facilmente da eventuali errori.
- Guida all'utilizzatore: messaggi significativi e comprensibili per l'utilizzatore, in caso di errori di quest'ultimo
- Diversità per gli utilizzatori: evidenzia che l'interfaccia deve essere progettata considerando il profilo reale degli utenti finali.

Questioni chiave dell'interfaccia

Presentati i principali modi con cui gli utenti interagiscono con il sistema. L'interazione consiste nel selezionare servizi, inserire dati e ricevere risposte dal software. Vengono presentati diversi stili di interazione.

- Quesiti-risposte: Il sistema comunica con l'utente attraverso uno scambio di domande e risposte: l'utente invia una richiesta e il sistema restituisce una risposta.
- Manipolazione diretta: l'utente interagisce direttamente con gli elementi sullo schermo tramite mouse, touch o altri dispositivi di puntamento.
- Selezione da menu: l'utente sceglie un comando da un elenco di opzioni proposte dal sistema.
- Compilazione di moduli: l'utente inserisce dati in campi specifici e il sistema esegue operazioni basate sui valori immessi.
- Linguaggio a comandi: l'utente scrive comandi testuali con eventuali parametri per dire al sistema quale operazione eseguire.
- Linguaggio naturale: l'utente comunica usando il linguaggio umano normale e il sistema interpreta la frase traducendola in operazioni eseguibili.
- Rappresentazione dell'informazione: Il sistema restituisce informazioni in forma:
testuale,

tabellare,
grafica,
a seconda del tipo e della quantità di dati da mostrare.

- Tempo di risposta: indica quanto velocemente il sistema risponde alle richieste dell'utente.
- Retroazione del sistema: il sistema fornisce feedback all'utente per confermare dati inseriti, operazioni eseguite o eventuali errori.
- Rappresentazioni astratte: servono a sintetizzare grandi quantità di informazioni attraverso grafici, dashboard o riepiloghi per facilitare la comprensione.
- Uso dei colori: i colori vengono utilizzati per evidenziare informazioni e migliorare leggibilità e comprensione dell'interfaccia.
- Analisi dell'utilizzatore: serve a studiare caratteristiche, abilità, compiti e ambiente degli utenti per progettare un'interfaccia adatta alle loro esigenze.
- Prototipazione: consiste nella creazione di versioni preliminari dell'interfaccia per testarla e raccogliere feedback prima dello sviluppo definitivo.
- Valutazione dell'interfaccia: serve a raccogliere dati sull'esperienza d'uso per migliorare continuamente usabilità, efficienza e qualità dell'interfaccia.

VERIFICA & VALIDAZIONE

La verifica controlla conformità e correttezza del progetto.
La validazione controlla che il sistema soddisfi i requisiti richiesti.

Le tecniche principali sono:

- Ispezione: validare il modello
- Prototipi: validare comportamenti più critici
- test di integrazione: validare che ogni componente interagisca con altre correttamente,
- test di sistema: verificare che i requisiti vengano soddisfatti.